Assignment No. 8

Subject: Database Engineering Lab (7CS271)
Class: SY CSE, AY 2025–26, Sem-II

Name - Atharv Vinayak Kulkarni

PRN - 245109074

Batch - S5

Indexing and Hashing

Q. Implement B+ Tree and Hash Indexing techniques using a programming language

Part A:

B+ Tree Implementation Implement a B+ Tree of order ‘m’.
Support the following operations:

- Insertion of keys
- Searching for a key
- Traversal (leaf-level linked traversal)

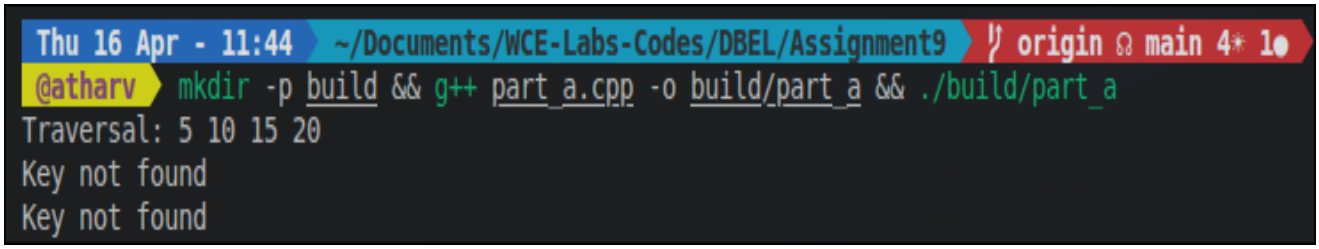
Code :

part_a.cpp > ...

```cpp
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

#define MAX 3  // order of tree

class Node {
public:
    bool isLeaf;
    vector<int> keys;
    vector<Node*> ptr;
    Node* next;

    Node(bool leaf) {
        isLeaf = leaf;
        next = NULL;
    }
};

class BPlusTree {
    Node* root;

public:
    BPlusTree() {
        root = new Node(true);
    }

    void insert(int key) {
        Node* curr = root;

        if (curr->keys.size() < MAX) {
            curr->keys.push_back(key);
            sort(curr->keys.begin(), curr->keys.end());
        } else {
            Node* newLeaf = new Node(true);
            vector<int> temp = curr->keys;
            temp.push_back(key);
            sort(temp.begin(), temp.end());

            curr->keys.clear();
            for (int i = 0; i < (MAX+1)/2; i++)
                curr->keys.push_back(temp[i]);

            for (int i = (MAX+1)/2; i < temp.size(); i++)
                newLeaf->keys.push_back(temp[i]);
```

```
47
48                  newLeaf->next = curr->next;
49                  curr->next = newLeaf;
50              }
51          }
52
53          void search(int key) {
54              Node* curr = root;
55
56              for (int k : curr->keys) {
57                  if (k == key) {
58                      cout << "Key found\n";
59                      return;
60                  }
61              }
62              cout << "Key not found\n";
63          }
64
65          void traverse() {
66              Node* curr = root;
67              while (curr != NULL) {
68                  for (int k : curr->keys)
69                      cout << k << " ";
70                  curr = curr->next;
71              }
72              cout << endl;
73          }
74      };
75
76      int main() {
77          BPlusTree tree;
78
79          tree.insert(10);
80          tree.insert(20);
81          tree.insert(5);
82          tree.insert(15);
83
84          cout << "Traversal: ";
85          tree.traverse();
86
87          tree.search(15);
88          tree.search(100);
89
90          return 0;
91      }
```

Output:


Thu 16 Apr - 11:44
~/Documents/WCE-Labs-Codes/DBEL/Assignment9
origin main 4* 1
@atharv
mkdir -p build && g++ part_a.cpp -o build/part_a && ./build/part_a
Traversal: 5 10 15 20
Key not found
Key not found

Part B:

Hash Indexing Implementation

Implement a hash table using: Division method: h(k) = k mod n Handle collisions using: Chaining OR Open Addressing (Linear Probing)

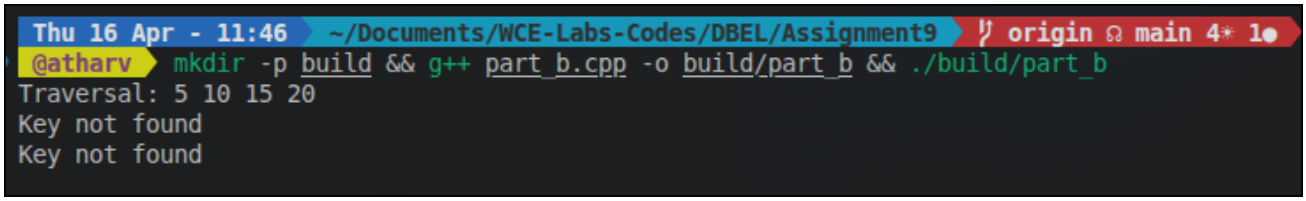
Code:

part_b.cpp > BPlusTree > insert(int)

```cpp
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

#define MAX 3  // order of tree

class Node {
public:
    bool isLeaf;
    vector<int> keys;
    vector<Node*> ptr;
    Node* next;

    Node(bool leaf) {
        isLeaf = leaf;
        next = NULL;
    }
};

class BPlusTree {
    Node* root;

public:
    BPlusTree() {
        root = new Node(true);
    }

    void insert(int key) {
        Node* curr = root;

        if (curr->keys.size() < MAX) {
            curr->keys.push_back(key);
            sort(curr->keys.begin(), curr->keys.end());
        } else {
            Node* newLeaf = new Node(true);
            vector<int> temp = curr->keys;
            temp.push_back(key);
            sort(temp.begin(), temp.end());

            curr->keys.clear();
            for (int i = 0; i < (MAX+1)/2; i++)
                curr->keys.push_back(temp[i]);

            for (int i = (MAX+1)/2; i < temp.size(); i++)
                newLeaf->keys.push_back(temp[i]);
```

```
47
48              newLeaf->next = curr->next;
49              curr->next = newLeaf;
50          }
51      }
52
53      void search(int key) {
54          Node* curr = root;
55
56          for (int k : curr->keys) {
57              if (k == key) {
58                  cout << "Key found\n";
59                  return;
60              }
61          }
62          cout << "Key not found\n";
63      }
64
65      void traverse() {
66          Node* curr = root;
67          while (curr != NULL) {
68              for (int k : curr->keys)
69                  cout << k << " ";
70              curr = curr->next;
71          }
72          cout << endl;
73      }
74  };
75
76  int main() {
77      BPlusTree tree;
78
79      tree.insert(10);
80      tree.insert(20);
81      tree.insert(5);
82      tree.insert(15);
83
84      cout << "Traversal: ";
85      tree.traverse();
86
87      tree.search(15);
88      tree.search(100);
89
90      return 0;
91  }
```

Output:


Thu 16 Apr - 11:46
~/Documents/WCE-Labs-Codes/DBEL/Assignment9
origin main 4 1
@atharv
mkdir -p build && g++ part_b.cpp -o build/part_b && ./build/part_b
Traversal: 5 10 15 20
Key not found
Key not found